

GHỊ CHÚ ĐẦY ĐỦ

JavaScript

KIẾN THỨC NỀN TẢNG • KHÁI NIỆM • VÍ DỤ • ÔN TẬP NHANH

“ Code hôm nay, chạy ngày mai. ”

CƠ BẢN

- Biến (Variables)
- Kiểu dữ liệu
- Toán tử
- Nhập / Xuất

THAO TÁC VỚI DOM

```
<h1>Hello</h1>
```

HÀM (FUNCTIONS)

```
function add(a, b) {  
  return a + b;  
}
```

CÂU LỆNH ĐIỀU KIỆN

if → Có/Không, rẽ nhánh xử lý theo điều kiện

SỰ KIỆN (EVENTS)

Click Me!

JS

JAVASCRIPT

NGÔN NGỮ CỦA WEB

MẢNG (ARRAYS)

```
[ 10, 20, 30, 40 ]  
Index → 0 1 2 3
```

ĐỐI TƯỢNG (OBJECTS)

```
{ name: "John",  
  age: 21,  
  role: "Developer" }
```

VÒNG LẶP (LOOPS)

```
for (let i=0; i<5; i++) {  
  console.log(i);  
}
```

ĐƠN GIẢN & DỄ HIỂU

GHỊ CHÚ VIẾT TAY

VÍ DỤ & ĐOẠN MÃ MÀU

ÔN TẬP NHANH

GHI CHÚ VIẾT TAY JAVASCRIPT

Phần 1

1. GIỚI THIỆU VỀ JAVASCRIPT

JavaScript là gì?

- JavaScript là một ngôn ngữ lập trình thông dịch (interpreted), nhẹ, được dùng để làm cho trang web trở nên tương tác và động. Đây là một trong những công nghệ cốt lõi của web, cùng với HTML và CSS.

Tại sao lại dùng JavaScript?

- Nó cho phép chúng ta thêm hành vi (behavior) vào trang web. Nó có thể cập nhật và thay đổi cả HTML lẫn CSS.

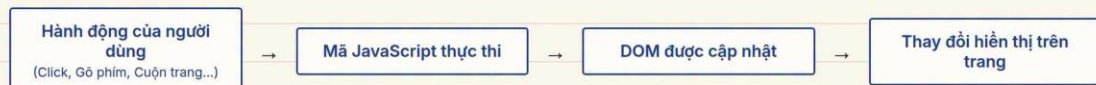
JavaScript chạy ở đâu?

- Trong trình duyệt (Client-side)
- Trên server bằng Node.js (Server-side)

Vai trò của JavaScript trong phát triển web



2. JAVASCRIPT HOẠT ĐỘNG NHƯ THẾ NÀO?



Lưu ý: DOM (Document Object Model) là cách biểu diễn cấu trúc HTML mà JavaScript có thể truy cập và thao tác.

3. ĐOẠN MÃ JAVASCRIPT ĐẦU TIÊN

```
<script>
  console.log("Hello, JavaScript!");
</script>
```

→ Đoạn mã này in ra dòng chữ trong console của trình duyệt.

Kết quả trong Console:

Hello, JavaScript!

→ Bạn có thể xem kết quả bằng cách mở console của trình duyệt.

4. TOÁN TỬ TRONG JAVASCRIPT

Toán tử (operator) được dùng để thực hiện các phép toán trên biến và giá trị.

Các loại toán tử:

Loại	Mô tả	Ví dụ
Số học (Arithmetic)	Dùng cho các phép toán học	+, -, *, /, %
Gán (Assignment)	Dùng để gán giá trị	=, +=, -=, *=, /=
So sánh (Comparison)	Dùng để so sánh hai giá trị	==, !=, >, <, >=, <=
Logic (Logical)	Dùng để kết hợp nhiều điều kiện	&&, , !
Một ngôi (Unary)	Dùng với một toán hạng duy nhất	++, --, !
Ba ngôi (Ternary)	Viết tắt của if...else	condition ? expr1 : expr2

Toán tử số học:

- + Cộng
- Trừ
- * Nhân
- / Chia
- % Chia lấy dư (Modulus)
- ++ Tăng (Increment)
- Giảm (Decrement)

Ví dụ:

```
let a = 10, b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
a++; // 11
b--; // 2
```

Toán tử so sánh:

- = Bằng
- ≠ Khác
- > Lớn hơn
- < Nhỏ hơn
- >= Lớn hơn hoặc bằng
- <= Nhỏ hơn hoặc bằng
- === Bằng tuyệt đối (giá trị & kiểu)
- !== Khác tuyệt đối (giá trị & kiểu)

Ví dụ:

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false (khác kiểu)
console.log(5 != 3); // true
console.log(5 > 3); // true
console.log(5 < 3); // false
console.log(5 >= 5); // true
console.log(5 <= 4); // false
```

Toán tử logic:

- && AND (VÀ)
- || OR (HOẶC)
- ! NOT (PHỦ ĐỊNH)

Ví dụ:

```
console.log(true && false); // false
console.log(true || false); // true
console.log(!true); // false
```

Toán tử ba ngôi (Ternary):

condition ? expr1 : expr2

Ví dụ:

```
let age = 18;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result); // Adult
```

1. BIẾN TRONG JAVASCRIPT

- Biến (variable) được dùng để lưu trữ dữ liệu.
- Trong JavaScript, ta có thể khai báo biến bằng **var**, **let** và **const**.

var	let	const
• Phạm vi hàm (Function scoped) • Có thể khai báo lại • Có thể cập nhật • Hoisting với giá trị undefined	• Phạm vi khối (Block scoped) • Không thể khai báo lại • Có thể cập nhật • Hoisting nhưng nằm trong Temporal Dead Zone (TDZ)	• Phạm vi khối (Block scoped) • Không thể khai báo lại • Không thể cập nhật • Phải khởi tạo giá trị ngay khi khai báo

Ví dụ:

```
var a = 10;
let b = 20;
const c = 30;

a = 15; // allowed
b = 25; // allowed
c = 35; // not allowed
```

Lưu ý: Nên ưu tiên dùng **let** và **const** thay vì **var**.

2. KIỂU DỮ LIỆU TRONG JAVASCRIPT

Kiểu dữ liệu (data type) dùng để xác định loại giá trị mà một biến có thể lưu trữ.

Kiểu dữ liệu		Ví dụ:
Nguyên thủy (7)	Không nguyên thủy / Tham chiếu	<pre>let name = "Aman"; // String let age = 22; // Number let isStudent = true; // Boolean let x; // Undefined let y = null; // Null let id = Symbol('id'); // Symbol let big = 123..890n; // BigInt let obj = {name: "Aman"}; // Object let arr = [1, 2, 3]; // Array function greet() {} // Function</pre>
• String • Number • Boolean • Undefined • Null • Symbol (ES6) • BigInt (ES2020)	• Object • Array • Function	

3. TOÁN TỬ typeof

Toán tử **typeof** dùng để kiểm tra kiểu dữ liệu của một biến.

```
console.log(typeof "Aman"); // string
console.log(typeof 22); // number
console.log(typeof true); // boolean
console.log(typeof undefined); // undefined
console.log(typeof null); // object
console.log(typeof {name: "Aman"}); // object
console.log(typeof [1, 2, 3]); // object
console.log(typeof function(){}); // function
```

Lưu ý:

- **typeof null** trả về "object" (đây là một lỗi đã biết của JavaScript).
- Cả Array và Object đều trả về "object".

5. CÂU LỆNH ĐIỀU KHIỂN

Các câu lệnh này dùng để điều khiển luồng thực thi của chương trình.

a) Câu lệnh if

```
if (condition) {  
  // code  
}
```

b) Câu lệnh if...else

```
if (condition) {  
  // code nếu true  
} else {  
  // code nếu false  
}
```

c) Chuỗi if...else if...else

```
if (condition1) {  
  // code  
} else if (condition2) {  
  // code  
} else {  
  // code  
}
```

d) Câu lệnh switch

```
switch (variable) {  
  case value1:  
    // code  
    break;  
  case value2:  
    // code  
    break;  
  default:  
    // code  
}
```

e) Vòng lặp (Loops)

• Vòng lặp for

```
for (initialization; condition; tăng/giảm){  
  // code  
}
```

• Vòng lặp while

```
while (condition) {  
  // code  
}
```

• Vòng lặp do...while

```
do {  
  // code  
} while (condition);
```

Luồng thực thi:

Condition

True ↓ False →

6. HÀM TRONG JAVASCRIPT

Hàm (function) là một khối mã được thiết kế để thực hiện một nhiệm vụ cụ thể.

Khai báo hàm (Function Declaration)

```
function greet(name) {  
  console.log("Hello, " + name);  
}  
greet("Aman"); // Gọi hàm
```

Biểu thức hàm (Function Expression)

```
const greet = function(name) {  
  console.log("Hello, " + name);  
};  
greet("Aman");
```

Arrow Function

```
const greet = (name) => {  
  console.log("Hello, " + name);  
};  
greet("Aman");
```

Lưu ý: Hàm giúp tái sử dụng mã và giữ cho code có tổ chức.

7. CHÚ THÍCH TRONG JAVASCRIPT

// Đây là chú thích một dòng.

```
/*  
Đây là chú thích  
nhiều dòng.  
*/
```

Ví dụ:

```
console.log(10); // in ra 10  
console.log(20); /* in ra 20 trên  
hai dòng chú thích */  
console.log(30); // in ra 30
```

8. MẢNG TRONG JAVASCRIPT

- Mảng (Array) là một loại object đặc biệt dùng để lưu nhiều giá trị trong một biến.
- Chỉ số (index) của mảng bắt đầu từ 0.

Ví dụ:

```
let fruits = ["Apple", "Banana", "Mango"];
0           1           2
index →    [Apple] [Banana] [Mango]
```

Truy cập phần tử:

- fruits[0] → Apple
- fruits[1] → Banana
- fruits[2] → Mango

Các phương thức mảng phổ biến:

- push()** → thêm phần tử vào cuối
- pop()** → xóa phần tử cuối cùng
- shift()** → xóa phần tử đầu tiên
- unshift()** → thêm phần tử vào đầu
- length** → trả về số lượng phần tử
- indexOf()** → trả về chỉ số của phần tử

Ví dụ:

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.push("Orange"); // [..., "Orange"]
fruits.pop();           // ["Apple", "Banana", "Mango"]
console.log(fruits[1]); // Banana
```

9. ĐỐI TƯỢNG TRONG JAVASCRIPT

- Object là một tập hợp các cặp key-value.
- Dùng để lưu trữ dữ liệu liên quan và các thực thể phức tạp hơn.

Ví dụ:

```
let person = {
  name: "Aman",
  age: 22,
  city: "Delhi"
};
```

Truy cập thuộc tính đối tượng:

- person.name → Aman
- person["age"] → 22
- person.city → Delhi

```
let person = { name: "Aman", age: 22, city: "Delhi" };
console.log(person.name); // Aman
console.log(person["age"]); // 22
```

10. CHUYỂN ĐỔI KIỂU DỮ LIỆU

- JavaScript tự động chuyển đổi kiểu dữ liệu này sang kiểu khác.
- Có hai loại: Chuyển đổi ngầm định (Implicit) và Chuyển đổi tường minh (Explicit).

Chuyển đổi ngầm định (tự động):

- "5" + 2 → "52" (string)
- "5" - 2 → 3 (number)
- true + 1 → 2 (number)
- false + 1 → 1 (number)
- null + 1 → 1 (number)
- undefined + 1 → NaN (Not a Number)

Chuyển đổi tường minh (thủ công):

- String → Number: Number("123") → 123
- Number → String: String(123) → "123"
- Boolean → String: String(true) → "true"
- String → Number: parseInt("45") → 45
- parseFloat("45.67") → 45.67

```
let a = "10";
console.log(typeof a); // string
console.log(typeof Number(a)); // number
```

```
let b = 20;
console.log(typeof String(b)); // string
console.log(typeof (a + b)); // string ("1020")
```

11. PHƯƠNG THỨC CHUỖI (STRING)

Chuỗi (string) được dùng để biểu diễn và thao tác với văn bản.

Các phương thức chuỗi phổ biến:

Phương thức	Mô tả
length	Trả về độ dài chuỗi
toUpperCase()	Chuyển chuỗi thành chữ hoa
toLowerCase()	Chuyển chuỗi thành chữ thường
trim()	Xóa khoảng trắng ở hai đầu
slice(start, end)	Trích một phần của chuỗi
replace(old, new)	Thay thế một giá trị trong chuỗi
includes(value)	Kiểm tra chuỗi có chứa giá trị không

Ví dụ:

```
let msg = "Hello JavaScript";
console.log(msg.length);           // 16
console.log(msg.toUpperCase());    // HELLO...
console.log(msg.toLowerCase());    // hello...
console.log(msg.trim());
console.log(msg.slice(6,10));      // Java
console.log(msg.replace("JavaScript", "JS"));
console.log(msg.includes("Java")); // true
```

12. TEMPLATE LITERALS (CHUỖI MẪU)

Template literals cho phép nhúng biểu thức và viết chuỗi nhiều dòng.

```
let name = "Aman";
let age = 22;
let msg = `
  Hello, ${name}!
  You are ${age} years old.
`;
console.log(msg);
```

Kết quả:

```
Hello, Aman!

You are 22 years old.
```

Lưu ý:

Dùng dấu backtick (') thay vì dấu nháy đơn hoặc kép.
Dùng \${ } để nhúng biến hoặc biểu thức.

13. TOÁN TỬ ĐIỀU KIỆN (TERNARY)

Cách viết tắt của câu lệnh if...else.

```
condition ? expression_if_true : expression_if_false
```

```
let age = 18;
let result = (age >= 18) ? "Adult" : "Minor";
console.log(result); // Adult
```

14. SỰ KIỆN TRONG JAVASCRIPT

- Sự kiện (event) là các hành động hoặc điều xảy ra trong trình duyệt.
- JavaScript có thể được dùng để xử lý các sự kiện này.

click

Xảy ra khi một phần tử được click.

mouseover

Xảy ra khi con trỏ chuột di chuyển vào.

mouseout

Xảy ra khi con trỏ chuột di chuyển ra.

keydown

Xảy ra khi một phím được nhấn.

submit

Xảy ra khi một form được gửi.

Ví dụ (sự kiện click):

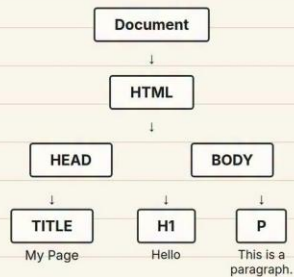
```
document.getElementById("btn").
  addEventListener("click", function() {
    alert("Button Clicked!");
  });
```

HTML: <button id="btn">Click Me</button>

15. DOM (DOCUMENT OBJECT MODEL)

- DOM biểu diễn cấu trúc của một tài liệu HTML dưới dạng cây (tree).
- JavaScript có thể truy cập và cập nhật nội dung, cấu trúc và kiểu dáng của trang thông qua DOM.

Ví dụ:



Truy cập phần tử DOM:

- `document.getElementById()` → theo id
- `document.getElementsByClassName()` → theo class
- `document.getElementsByTagName()` → theo tag
- `document.querySelector()` → theo css selector
- `document.querySelectorAll()` → tất cả phần tử khớp

Ví dụ:

```
<h1 id="title">Hello DOM</h1>
<p class="text">Welcome</p>

let h1 = document.getElementById("title");
let p = document.getElementsByClassName("text")[0];
h1.innerText = "Hello JavaScript";
p.innerText = "Welcome to JS";
```

16. XỬ LÝ SỰ KIỆN

- Sự kiện (event) là các hành động xảy ra trong trình duyệt.
- Chúng ta có thể phản hồi sự kiện bằng cách dùng event listener.

Các sự kiện phổ biến:

Sự kiện	Mô tả
click	Xảy ra khi một phần tử được click
mouseover	Xảy ra khi chuột di chuyển vào phần tử
mouseout	Xảy ra khi chuột di chuyển ra khỏi phần tử
keydown	Xảy ra khi một phím được nhấn
submit	Xảy ra khi một form được gửi

Ví dụ (sự kiện click):

```
<button id="btn">Click Me</button>
<p id="msg"></p>

document.getElementById("btn").addEventListener
("click", function() {
    document.getElementById("msg").innerText =
        "Button Clicked!";
});
```

17. CHÚ THÍCH TRONG JAVASCRIPT

a) Chú thích một dòng

```
// Đây là chú thích một dòng
```

b) Chú thích nhiều dòng

```
/*
    Đây là chú thích
    nhiều dòng
*/
```

Lưu ý:

Chú thích được JavaScript bỏ qua và dùng để giải thích code.

18. PHẠM VI (SCOPE) TRONG JAVASCRIPT

- Scope xác định khả năng truy cập (visibility) của biến.
- JavaScript có 3 loại phạm vi.

Loại phạm vi		Mô tả
1. Global Scope	—	Khai báo bên ngoài mọi hàm, truy cập được ở mọi nơi.
2. Function Scope	—	Khai báo bên trong một hàm, chỉ truy cập được trong hàm đó.
3. Block Scope	—	Khai báo bên trong { } bằng let hoặc const, chỉ truy cập được trong khối đó.

1. Global Scope

```
let a = 10; // global
function test() {
  console.log(a); // 10
}
test();
console.log(a); // 10
```

2. Function Scope

```
function test() {
  let b = 20; // function scope
  console.log(b); // 20
}
test();
// console.log(b); // Lỗi
```

3. Block Scope (let/const)

```
if (true) {
  let c = 30; // block scope
  console.log(c); // 30
}
// console.log(c); // Lỗi
```

19. KIỂU DỮ LIỆU TRONG JAVASCRIPT

JavaScript có 7 kiểu dữ liệu nguyên thủy và 1 kiểu dữ liệu không nguyên thủy (Object).

Kiểu dữ liệu nguyên thủy (7):

- 1. String → "Hello"
- 2. Number → 123, 45.67
- 3. Boolean → true, false
- 4. Undefined → let a;
- 5. Null → let a = null;
- 6. Symbol → let s = Symbol("id");
- 7. BigInt → let n = 9007199254740991n;

Kiểu dữ liệu không nguyên thủy (1):

- Object → { key: "value" }

Ví dụ:

```
let person = {
  name: "Aman",
  age: 22
};
console.log(typeof person); // object
```

20. TOÁN TỬ TYPEOF

Toán tử typeof dùng để kiểm tra kiểu dữ liệu của một giá trị.

Giá trị	typeof Giá trị	Kết quả
"Hello"	typeof "Hello"	"string"
123	typeof 123	"number"
true	typeof true	"boolean"
undefined	typeof undefined	"undefined"

Giá trị	typeof Giá trị	Kết quả
null	typeof null	"object" (trường hợp đặc biệt)
Symbol("id")	typeof Symbol("id")	"symbol"
900n	typeof 900n	"bigint"
{ }	typeof { }	"object"

21. BIẾN TRONG JAVASCRIPT

- Biến (variable) được dùng để lưu trữ dữ liệu.
- JavaScript có 3 cách khai báo biến.

1. var (cách cũ)
var a = 10;
// function scoped

2. let (cách hiện đại)
let b = 20;
// block scoped

3. const (hằng số)
const c = 30;
// cannot be reassigned

22. BIẾN

- Biến (variable) là các "hộp chứa" (container) dùng để lưu trữ dữ liệu.
- Quy tắc đặt tên biến:

1. Phải bắt đầu bằng chữ cái, \$ hoặc _
2. Không được bắt đầu bằng số.
3. Có thể chứa chữ cái, số, \$ và _
4. Phân biệt hoa thường (age và Age là khác nhau)
5. Không được là từ khóa (keyword) của JavaScript.

Ví dụ:

Biến hợp lệ	Biến không hợp lệ
name	1name (bắt đầu bằng số)
_name	user-name (chứa dấu -)
\$price	let (từ khóa)
age1	first name (chứa khoảng trắng)
user_name	@value (chứa @)

23. TOÁN TỬ SỐ HỌC

Toán tử	Tên	Ví dụ	Kết quả
+	Cộng	5 + 3	8
-	Trừ	5 - 3	2
*	Nhân	5 * 3	15
/	Chia	10 / 2	5
%	Chia lấy dư	10 % 3	1
++	Tăng	a++	a + 1
--	Giảm	a--	a - 1

Ví dụ:

```
let a = 10, b = 3;
console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
a++;
console.log(a); // 11
b--;
console.log(b); // 2
```

24. TOÁN TỬ SO SÁNH

Toán tử	Tên	Ví dụ	Kết quả
==	Bằng	5 == 5	true
!=	Khác	5 != 3	true
===	Bằng tuyệt đối	5 === "5"	false
!==	Khác tuyệt đối	5 !== "5"	true
>	Lớn hơn	5 > 3	true
<	Nhỏ hơn	5 < 3	false
>=	Lớn hơn hoặc bằng	5 >= 5	true
<=	Nhỏ hơn hoặc bằng	5 <= 4	false

Ví dụ:

```
let x = 5, y = "5";
console.log(x == y); // true
console.log(x === y); // false
console.log(x != y); // false
console.log(x !== y); // true
console.log(x > 3); // true
console.log(x <= 5); // true
```

25. TOÁN TỬ LOGIC

Toán tử	Tên	Mô tả	Ví dụ
&&	AND	Trả về true nếu cả hai điều kiện đều đúng	(5>3 && 3>1)
	OR	Trả về true nếu ít nhất một điều kiện đúng	(5>3 3<1)
!	NOT	Đảo ngược kết quả	!(5>3)

Ví dụ:

```
let a = 5, b = 3, c = 1;
console.log(a > b && b > c); // true
console.log(a < b || c < b); // true
console.log(!(a < b)); // true
```

26. CÂU LỆNH ĐIỀU KHIỂN LƯỖNG

Các câu lệnh điều khiển luồng dùng để điều khiển việc thực thi dựa trên điều kiện.

1. Câu lệnh if

Thực thi một khối nếu điều kiện đúng.

2. if...else

Thực thi một khối nếu đúng, khối khác nếu sai.

3. Chuỗi else if

Kiểm tra nhiều điều kiện lần lượt.

4. Câu lệnh switch

Thực thi một khối dựa trên các trường hợp khác nhau.

5. Toán tử ternary

Cách viết tắt của if...else cho điều kiện đơn giản.

Vi dụ:

```
let x = 10;
if (x>5) {
  console.log(
    "X>5");
}
```

```
let x = 3;
if (x>5) {
  .."X>5"
} else {
  .."X<=5"
}
```

```
let x = 7;
if (x>10) {..}
else if (x>5) {
  .."GT 5"
} else {..}
```

```
let day = 2;
switch (day) {
  case 1: ..break;
  case 2: ..break;
  default: ..
}
```

```
let age=18;
let result =
  (age>=18)
  ? "Adult"
  : "Minor";
```

27. VÒNG LẶP TRONG JAVASCRIPT

- Vòng lặp (loop) dùng để thực thi một khối mã nhiều lần.
- JavaScript có 3 loại vòng lặp.

1. Vòng lặp for

Dùng khi biết trước số lần lặp.

```
for (let i=1; i<=5; i++) {
  console.log(i); // 1 đến 5
}
```

2. Vòng lặp while

Thực thi khối lệnh khi điều kiện còn đúng.

```
let i=1;
while (i<=5) {
  console.log(i);
  i++;
}
```

3. Vòng lặp do...while

Thực thi khối lệnh ít nhất một lần, rồi mới kiểm tra điều kiện.

```
let i=1;
do {
  console.log(i);
  i++;
} while (i<=5);
```

28. HÀM TRONG JAVASCRIPT

- Hàm (function) là một khối mã được thiết kế để thực hiện một nhiệm vụ cụ thể.
- Giúp tái sử dụng và tổ chức mã tốt hơn.

1. Khai báo hàm

```
function add(a, b) {
  return a + b;
}
let sum = add(5, 3);
console.log(sum); // 8
```

2. Biểu thức hàm

```
let multiply = function(a, b) {
  return a * b;
};
let result = multiply(4, 2);
console.log(result); // 8
```

3. Arrow Function (ES6)

```
let divide = (a, b) => a / b;
let res = divide(10, 2);
console.log(res); // 5
```

4. Tham số mặc định

```
function greet(name = "Guest") {
  console.log("Hello, "+name);
}
greet(); // Hello, Guest
greet("Aman"); // Hello, Aman
```

5. Tham số Rest (ES6)

```
function sumAll(...nums) {
  return nums.reduce((a,b)=>a+b, 0);
}
console.log(sumAll(1,2,3,4)); // 10
```

6. Hàm tự gọi (IIFE)

```
(function() {
  console.log("IIFE executed");
})();
// Chạy ngay lập tức
```

29. HÀM TRONG JAVASCRIPT

- Hàm (function) là các khối mã có thể tái sử dụng để thực hiện một nhiệm vụ cụ thể.
- Chúng giúp mã nguồn dễ tái sử dụng và có tính module hóa.

Cú pháp hàm:

```
function functionName(parameters) {  
  // code cần thực thi  
  return value; // tùy chọn  
}
```

Ví dụ:

```
function add(a, b) {  
  let sum = a + b;  
  return sum;  
}  
let result = add(5, 3);  
console.log(result); // 8
```

Cách hoạt động:

- Định nghĩa hàm bằng từ khóa function.
- Truyền tham số (argument) khi gọi hàm.
- Mã bên trong hàm được thực thi.
- Nếu dùng return, giá trị sẽ được trả về.

Các loại hàm:

1. Khai báo hàm

```
function greet(name) {  
  return "Hello " + name;  
}  
console.log(greet("Aman"));  
// Hello Aman
```

- Định nghĩa bằng từ khóa function.
- Có thể gọi trước khi khai báo.

2. Biểu thức hàm

```
const greet = function(name) {  
  return "Hello " + name;  
};  
console.log(greet("Aman"));  
// Hello Aman
```

- Được gán cho một biến.
- Không thể gọi trước khi khai báo.

3. Arrow Function (ES6)

```
const greet = (name) => {  
  return "Hello " + name;  
};  
console.log(greet("Aman"));  
// Hello Aman
```

- Cú pháp ngắn gọn hơn.
- Không có this riêng.

Parameters vs Arguments:

- Parameter là biến được liệt kê trong phần định nghĩa hàm.
- Argument là giá trị được truyền vào khi gọi hàm.

Parameters (khi định nghĩa)

```
function multiply(x, y) {  
  return x * y;  
}
```

Arguments (khi gọi)

```
let result = multiply(4, 5);  
console.log(result); // 20
```

Câu lệnh return:

- return dùng để gửi trả một giá trị từ hàm.
- Nếu không dùng return, hàm trả về undefined.

```
function square(n) {  
  return n * n;  
}  
console.log(square(4));  
// 16
```

```
function print() {  
  console.log("Hi");  
}  
let val = print();  
console.log(val); // undefined
```

Tham số mặc định (ES6):

```
function greet(name = "Guest") {  
  return "Hello, " + name;  
}  
console.log(greet()); // Hello, Guest  
console.log(greet("Aman")); // Hello, Aman
```

Tham số Rest (ES6):

```
function sumAll(...nums) {  
  return nums.reduce((acc, val) => acc + val, 0);  
}  
console.log(sumAll(1, 2, 3)); // 6  
console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

30. HÀM TRONG JAVASCRIPT (TIẾP THEO)

- Hàm có thể được tạo theo nhiều cách khác nhau.
- Chúng có thể có tham số, giá trị trả về và được dùng nhiều lần.

Các cách khác để tạo hàm:

1. Biểu thức hàm

```
const add = function(a, b) {  
  return a + b;  
}  
console.log(add(2, 3)); // 5
```

2. Arrow Function (ES6)

```
const add = (a, b) => {  
  return a + b;  
}  
console.log(add(2, 3)); // 5
```

3. Arrow Function (rút gọn)

```
const add = (a, b) => a + b;  
  
console.log(add(2, 3)); // 5
```

4. Hàm tự gọi (Immediately Invoked Function Expression - IIFE)

```
(function() {  
  console.log("IIFE executed");  
})();  
// Chạy ngay lập tức
```

Hàm với tham số mặc định (ES6):

```
function greet(name = "Guest") {  
  return "Hello, " + name;  
}  
console.log(greet()); // Hello, Guest  
console.log(greet("Aman")); // Hello, Aman
```

Hàm với tham số Rest (ES6):

```
function sumAll(...nums) {  
  return nums.reduce((acc, val) => acc + val, 0);  
}  
console.log(sumAll(1, 2, 3)); // 6  
console.log(sumAll(1, 2, 3, 4, 5)); // 15
```

Câu lệnh return:

- return dùng để gửi trả một giá trị từ hàm.
- Nếu không dùng return, hàm trả về undefined.

```
function square(n) {  
  return n * n;  
}  
console.log(square(4)); // 16
```

```
function show() {  
  console.log("Hi");  
}  
let val = show();  
console.log(val); // undefined
```

Quy tắc / Lưu ý về hàm:

- Tên hàm nên tuân theo quy tắc đặt tên biến.
- Tên hàm phân biệt hoa thường.
- Hàm có thể được truyền như một giá trị.
- Hàm có thể trả về một hàm khác.
- Có thể sử dụng hàm ẩn danh (anonymous function).

Ví dụ: Hàm trả về một hàm khác

```
function outer() {  
  return function inner() {  
    return "Hello from inner";  
  }  
}  
const fn = outer();  
console.log(fn()); // Hello from inner
```